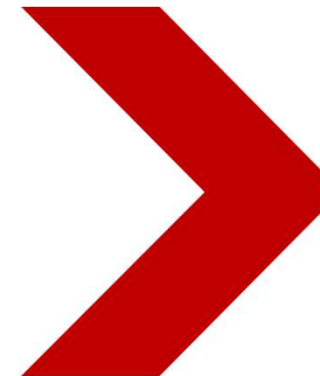


Java程序设计基础

Java Programming Fundamentals

Classes and Objects(1)





目录

- 面向对象的特征
- 类与对象
- 匿名对象

面向过程→面向对象

- 面向过程：
 - 数据与对数据的处理过程相分离
 - 可维护、可复用性和可扩展性较差
- 面向对象：
 - 将数据和对数据的处理封装起来，模块耦合度低
 - 支持继承、多态，可维护、可复用性和可扩展性提升，



面向对象的三大特征

- 封装 (Encapsulation)
 - 对外部不可见
- 继承 (Inheritance)
 - 扩展类的功能
- 多态 (Polymorphism)
 - 方法的重载
 - 对象的多态性



目录

- 面向对象的特征
- 类与对象
- 匿名对象

类图

- 类图的层次

- 第一层：类名
- 第二层：属性 定义格式：访问权限 属性名称:属性类型
- 第三层：方法 定义格式：访问权限 方法名称():方法返回值

- IDEA中支持将源码转换成类图

- IDEA打开源文件，右键
- 选择Diagrams>Show Diagram...
- 将Java源代码转换成类图

Person	
- name	: String
- age	: int
+ tell ()	: void
+ getName ()	: String
+ setName (String n)	: void
+ getAge ()	: int
+ setAge (int a)	: void

对象的创建及使用

- 类名 对象名称 = null; // 声明对象
- 对象名称 = new 类名(); // 实例化对象
- 类名 对象名称 = new 类名();

```
class Person {  
    String name;  
    int age;  
    public void tell() {  
        System.out.println("姓名: " + name + ", 年龄: " + age);  
    }  
}  
  
public class ClassDemo02 {  
    public static void main(String args[]){  
        Person per = new Person() ;//创建并实例化对象  
    }  
}
```

访问类中的成员变量和方法

- 访问属性：对象名称.成员变量名
- 访问方法：对象名称.成员方法名()

```
class Person {  
    String name;  
    int age;  
    public void tell() {  
        System.out.println("姓名: " + name + ", 年龄: " + age);  
    }  
}  
  
public class ClassDemo03 {  
    public static void main(String args[]){  
        Person per = new Person() ;  
        per.name = "张三" ;           // 为成员变量赋值  
        per.age = 30 ;  
        per.tell() ;                 // 调用类中的方法  
    }  
}
```

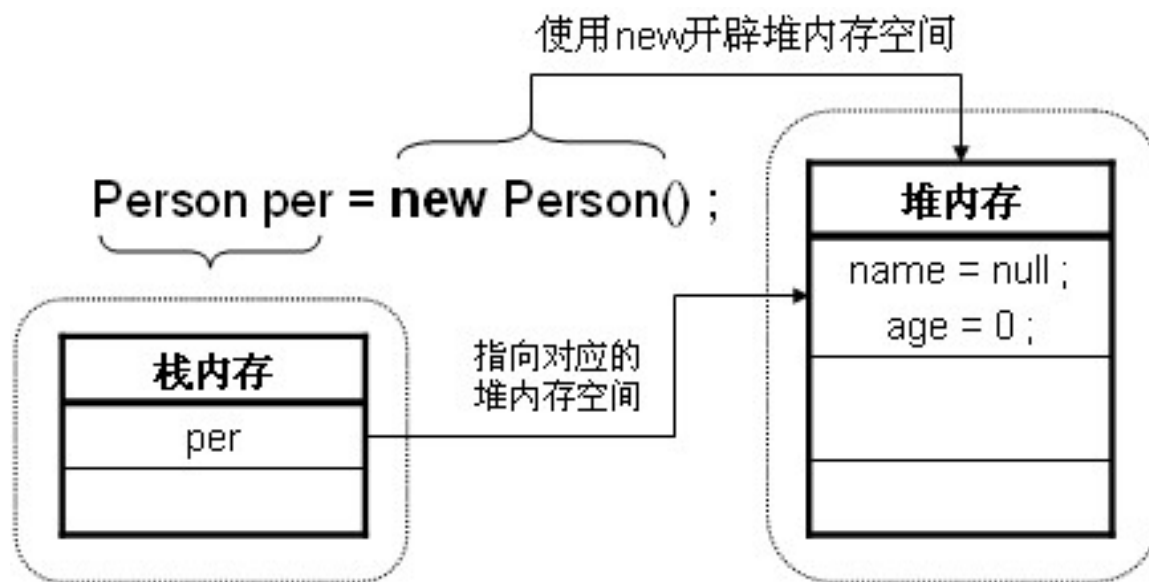



成员变量(属性) 与局部变量的区别

- 在类中的位置不同
 - 成员变量：在类中方法外
 - 局部变量：在方法中定义或者方法声明上
- 在内存中的位置不同
 - 成员变量：在堆内存中
 - 局部变量：在栈内存中
- 生命周期不同
 - 成员变量：随着对象的创建而存在，随着对象的消失而消失
 - 局部变量：随着方法的调用而存在，随着方法的调用完毕而消失
- 初始化值不同
 - 成员变量：有默认的初始化值
 - 局部变量：没有默认的初始化值，必须定义，赋值，然后才能使用
- 注意事项
 - 局部变量名称可以和成员变量名称一样，在方法中使用，采用就近原则

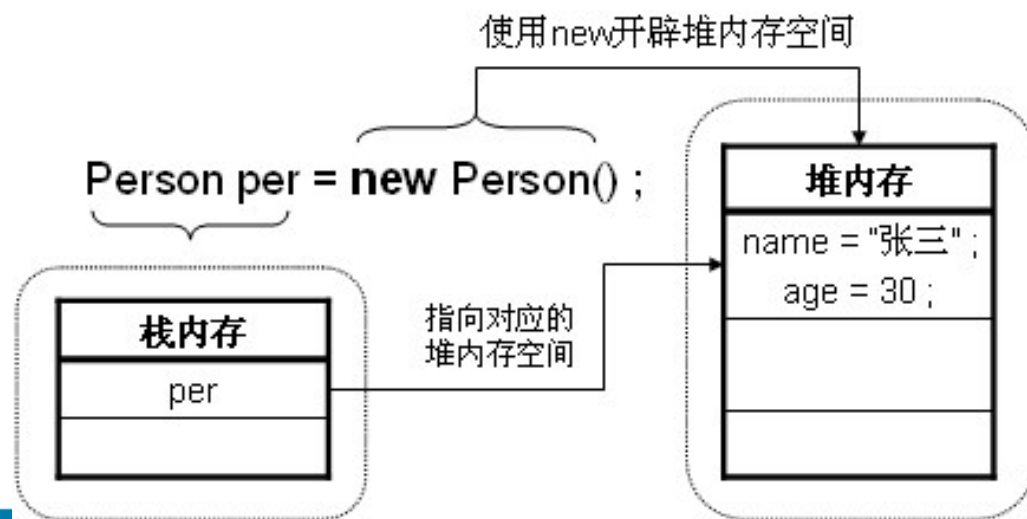
Java类的内存空间-初始创建

```
public class ClassDemo02 {  
    public static void main(String args[]){  
        Person per = new Person() ;  
    }  
}
```



Java类的内存空间-属性赋值

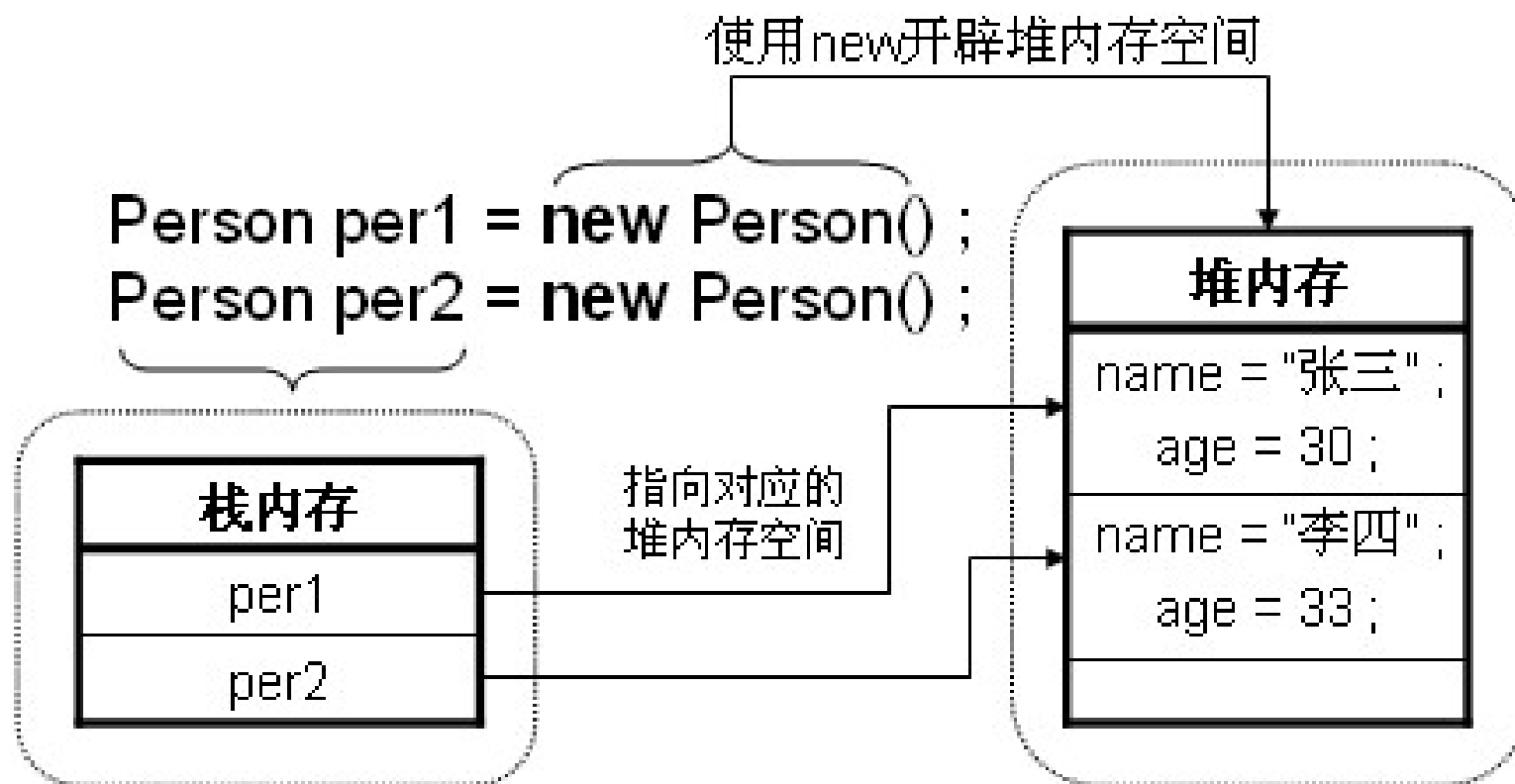
```
public class ClassDemo03 {  
    public static void main(String args[]){  
        Person per = new Person() ;  
        per.name = "张三" ;           // 为属性赋值  
        per.age = 30 ;  
        per.tell() ;                  // 调用类中的方法  
    }  
}
```



多个对象的内存视图

```
public class ClassDemo04 {  
    public static void main(String args[]) {  
        Person per1 = null;    // 声明per1对象  
        Person per2 = null;    // 声明per2对象  
        per1 = new Person();   // 实例化per1对象  
        per2 = new Person();   // 实例化per2对象  
        per1.name = "张三";    // 设置per1对象的名字属性内容  
        per1.age = 30;         // 设置per1对象的age属性内容  
        per2.name = "李四";    // 设置per2对象的名字属性内容  
        per2.age = 33;         // 设置per2对象的age属性内容  
        System.out.print("per1对象中的内容 --> ");  
        per1.tell();           // per1调用方法  
        System.out.print("per2对象中的内容 --> ");  
        per2.tell();           // per2调用方法  
    }  
}
```

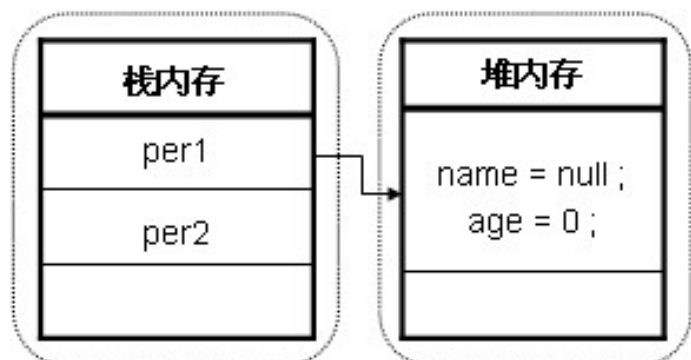
多个对象的内存视图



对象引用传递

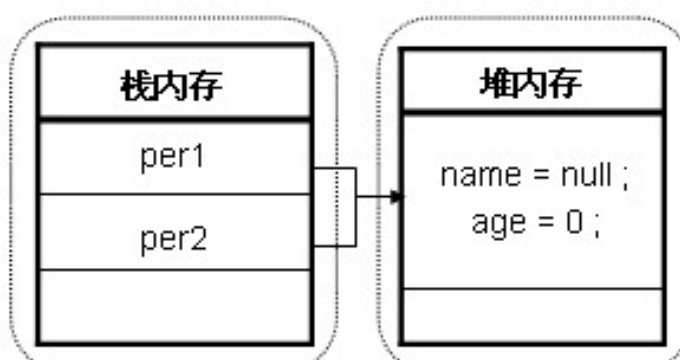
```
public class ClassDemo05 {  
    public static void main(String args[]) {  
        Person per1 = null;           // 声明per1对象  
        Person per2 = null;           // 声明per2对象  
        per1 = new Person();           // 只实例化per1一个对象  
        per2 = per1;                   // 把per1的堆内存空间使用权给per2  
        per1.name = "张三";           // 设置per1对象的名字属性内容  
        per1.age = 30;                 // 设置per1对象的age属性内容  
        // 设置per2对象的内容, 实际上就是设置per1对象的内容  
        per2.age = 33;  
        System.out.print("per1对象中的内容 --> ") ;  
        per1.tell();                   // 调用类中的方法  
        System.out.print("per2对象中的内容 --> ") ;  
        per2.tell();  
    }  
}
```

对象引用传递

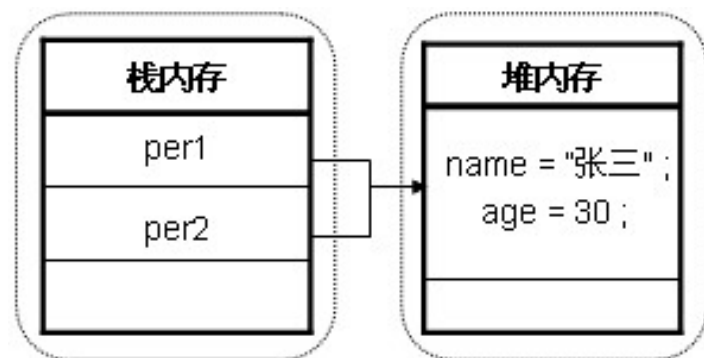


Person per1 = null;
Person per2 = null;

// 为per1开辟空间
per1 = new Person();

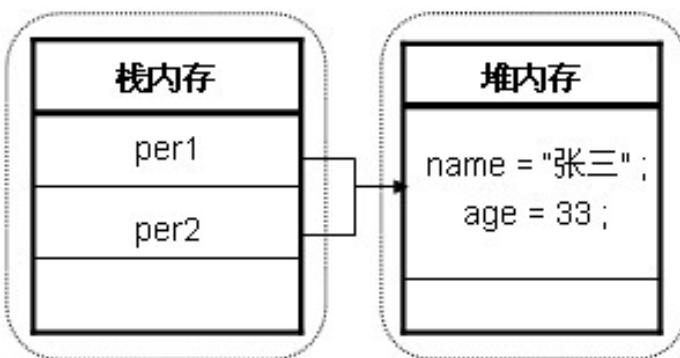


per2 = per1 ; // 此时per1和per2指向同一空间



per1.name = "张三";
per1.age = 30;

为per1中的属性赋值



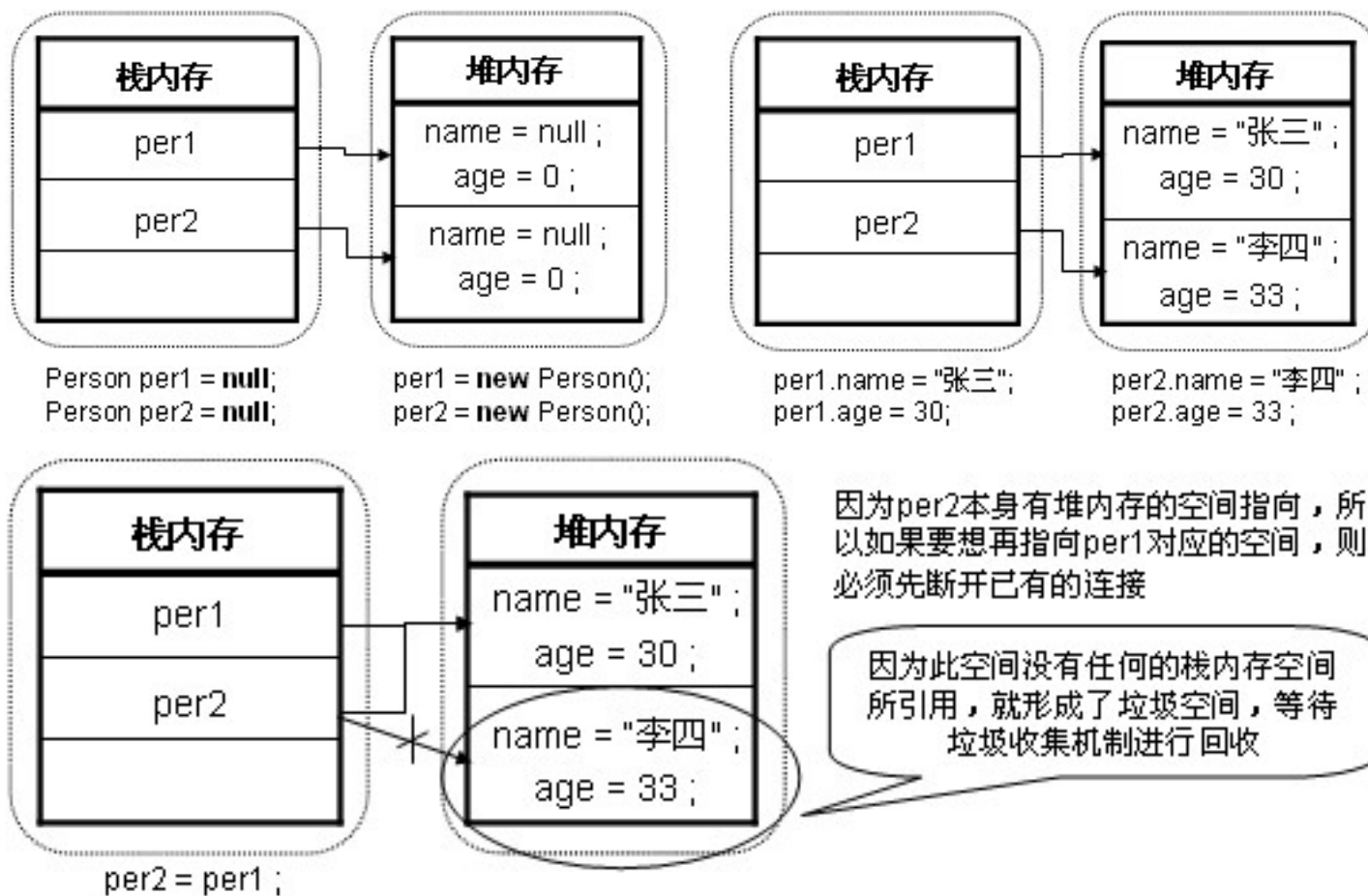
per2.age = 33;

per1和per2指向同一空间
所以修改对两个对象都有效

内存垃圾的产生

```
public class ClassDemo06 {  
    public static void main(String args[]) {  
        Person per1 = null;           // 声明一个per1对象  
        Person per2 = null;           // 声明一个per2对象  
        per1 = new Person();           // 实例化per1对象  
        per2 = new Person();           // 实例化per2对象  
        per1.name = "张三";            // 设置per1的name属性内容  
        per1.age = 30;                 // 设置per1的age属性内容  
        per2.name = "李四" ;           // 设置per2的name属性内容  
        per2.age = 33;                 // 设置per2的age属性内容  
        per2 = per1;                   // 将per1的引用传递给per2  
        System.out.print("per1对象中的内容 --> ") ;  
        per1.tell();                   // 调用类中的方法  
        System.out.print("per2对象中的内容 --> ") ;  
        per2.tell();  
    }  
}
```


内存垃圾的产生



属性封装及访问权限

```
class Person {  
    String name;           // 声明姓名属性  
    int age;               // 声明年龄属性  
    public void tell() {   // 取得信息的方法  
        System.out.println("姓名: " + name + ", 年龄: " + age);  
    }  
}  
  
public class EncDemo01 {  
    public static void main(String args[]) {  
        Person per = new Person(); // 声明并实例化对象  
        per.name = "张三";         // 为name属性赋值  
        per.age = -30;             // 为age属性赋值  
        per.tell();               // 调用方法  
    }  
}
```

封装及访问权限

```
class Person {  
    private String name;           // 声明姓名属性  
    private int age;               // 声明年龄属性  
    public void tell() {           // 取得信息的方法  
        System.out.println("姓名: " + name + ", 年龄: " + age);  
    }  
}  
  
public class EncDemo02 {  
    public static void main(String args[]) {  
        Person per = new Person();  
        per.name = "张三";         // 错误, 无法访问封装属性  
        per.age = -30;             // 错误, 无法访问封装属性  
        per.tell();  
    }  
}
```

为封装后的数据增加访问接口 (setter/getter)

```
class Person {  
    private String name;  
    private int age;  
    public void tell() {  
        System.out.println("姓名: " + getName() + ", 年龄: " + getAge());  
    }  
    public String getName() { //获取姓名属性  
        return name;  
    }  
    public void setName(String n) { //设置姓名属性  
        name = n;  
    }  
    public int getAge() {  
        return age;  
    }  
    public void setAge(int a) { //设置年龄时可加入验证  
        if (a >= 0 && a < 150) // 在此处加上验证代码  
            age = a;  
    }  
}
```

总结：属性封装步骤

• 属性封装实现步骤

- 1、对属性进行私有化（使用private进行修饰）
- 2、对外提供公开的setter/getter方法
- 3、如果该属性需要安全性的判断 将这些代码写在setter中

Person	
- name	: String
- age	: int
+ tell ()	: void
+ getName ()	: String
+ setName (String n)	: void
+ getAge ()	: int
+ setAge (int a)	: void

类的构造方法

- 构造方法

- 对象的产生格式：类名称 对象名称 = new 类名称()

- 构造方法的实现

```
class 类名称{  
    访问权限 类名称 (类型1 参数1, 类型2 参数2, ...) {  
        程序语句 ;  
        ... // 构造方法没有返回值  
    }  
}
```

- ✓ 构造方法的名称必须与类名称一致
- ✓ 构造方法的声明处不能有任何返回值类型的声明
- ✓ 不能在构造方法中使用return返回一个值

构造方法示例

```
class Person {  
    public Person() { // 声明构造方法  
        System.out.println("一个新的Person对象产生。") ;  
    }  
}  
  
public class ConsDemo01 {  
    public static void main(String args[]) {  
        System.out.println("声明对象: Person per = null ;") ;  
        Person per = null ; // 声明对象时不调用构造  
        System.out.println("实例化对象: per = new Person() ;") ;  
        per = new Person() ; // 实例化对象时调用构造  
    }  
}
```

默认构造方法

- 每个类中肯定都会有一个构造方法。
- 如果一个类中没有声明一个明确的构造方法则会自动生成一个无参的什么都不做的构造方法。

```
class Person {  
    public Person(){}  
}
```


构造方法的目的：属性初始化

```
class Person {  
    private String name;           // 声明姓名属性  
    private int age;              // 声明年龄属性  
    public Person(String name,int age){ // 定义构造方法为属性初始化  
        this.setName(name) ;      // 为name属性赋值  
        this.setAge(age) ;         // 为age属性赋值  
    }  
    public void tell() {           // 取得信息的方法  
        System.out.println("姓名: " + getName() + ", 年龄: " + getAge());  
    }  
    public void setAge(int a) {    // 设置年龄  
        if (a >= 0 && a < 150) {age = a; // 在此处加上验证代码  
        }  
    }  
}  
public class ConsDemo02 {  
    public static void main(String args[]) {  
        Person per = new Person("张三",30); //调用构造方法, 传递两个参数  
        per.tell();                        // 输出信息  
    }  
}
```

构造方法支持重载

```
public Person(){}  
public Person(String name){  
    this.setName(name) ;  
}  
public Person(String name,int age){  
    this.setName(name) ;  
    this.setAge(age) ;  
}
```



目录

- 面向对象的特征
- 类与对象
- 匿名对象

匿名对象

- 只使用一次的对象，称为匿名对象。
- 匿名对象只在堆内存中开辟空间，而不存在栈内存的引用。

```
public class NonameDemo01 {  
    public static void main(String args[]) {  
        Person per1=new Person("张三", 30);  
        per1.tell();  
        new Person("李四", 24).tell();           // 匿名对象  
    }  
}
```

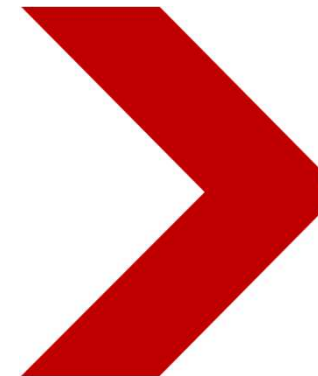
类的设计与实现总结

- 分析类所包含的属性;
- 对属性都必须进行封装 (private) ;
- 封装之后的属性通过setter和getter设置和取得;
- 如果需要可以加入若干构造方法;
- 再根据其他要求添加相应的方法;
- 类中的所有方法都不要直接输出, 而是交给被调用处输出。

Java程序设计基础

Java Programming Fundamentals

Classes and Objects(2)





目录

- 对象的引用传递
- this
- static
- main



引用传递

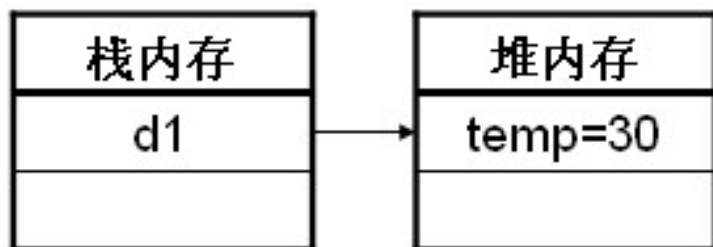
- 引用传递是将堆内存空间的使用权交给多个栈内存空间
- 三种方式



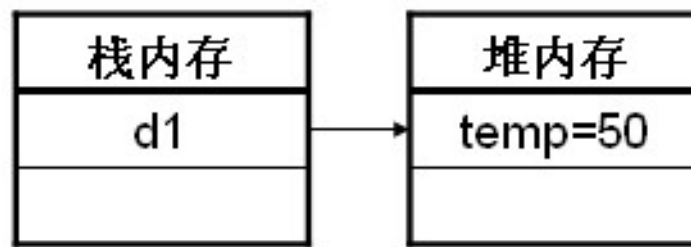
引用传递:将堆内存空间的使用权交给多个栈内存空间

```
class Demo {  
    int temp = 30;           // 此处为了访问方便, 属性暂不封装  
}  
  
public class RefDemo01{ //第一种引用传递  
    public static void main(String[] args) {  
        Demo d1 = new Demo();  
        d1.temp = 50;  
        System.out.println("fun() 方法调用之前: " + d1.temp);  
        fun(d1);  
        System.out.println("fun() 方法调用之后: " + d1.temp);  
    }  
    public static void fun(Demo d2) { // 此处的方法由主方法直接调用  
        d2.temp = 1000;  
    }  
}
```

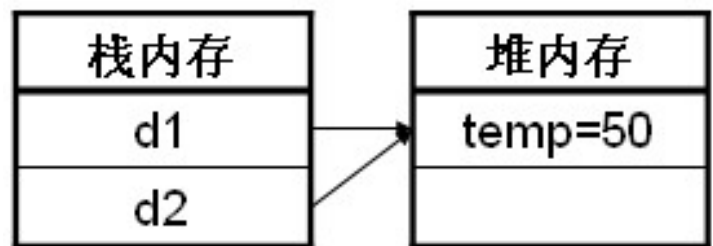
引用传递的内存分析



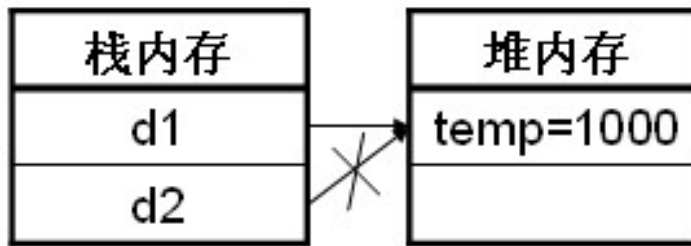
`Demo d1 = new Demo();`



`d1.temp = 50;`



`fun(d1);`

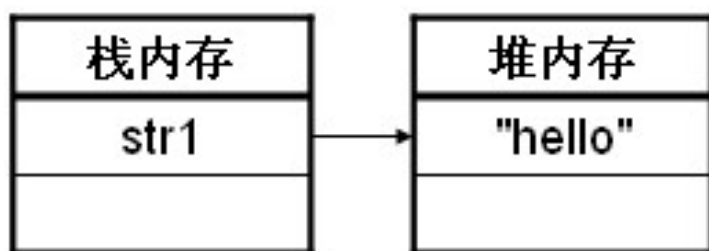


`fun()`方法执行完后断开连接

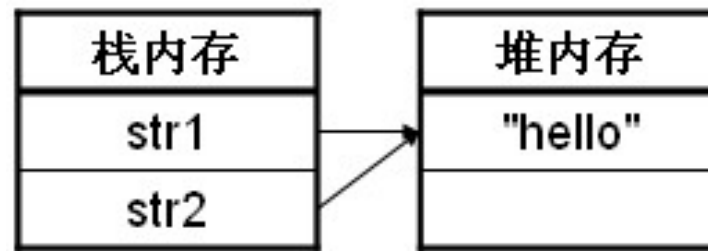
第2种引用传递

```
public class RefDemo02 {  
    public static void main(String[] args) {  
        String str1 = "hello" ;           // 实例化字符串对象  
        System.out.println("fun() 方法调用之前: " + str1);  
        fun(str1);                         // 调用fun() 方法  
        System.out.println("fun() 方法调用之后: " + str1);  
    }  
    public static void fun(String str2) {  // 此处的方法由主方法直接调用  
        str2 = "MLDN" ;                  // 修改字符串内容  
    }  
}
```

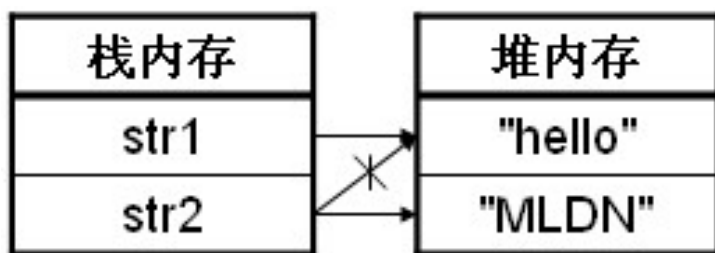
内存分析



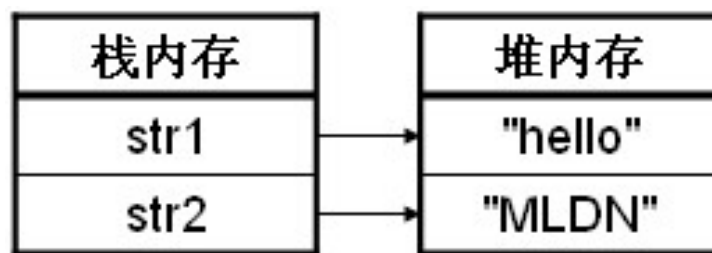
`String str1 = "hello";`



`fun(str1)`



`str2 = "MLDN";`

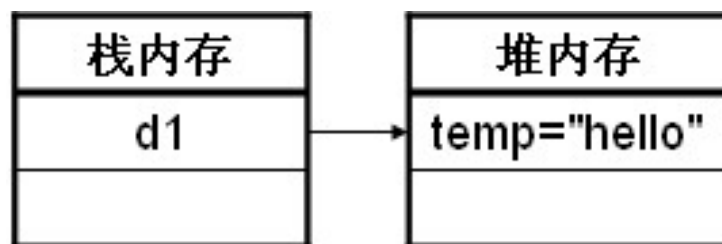


`fun()`方法执行完后断开连接

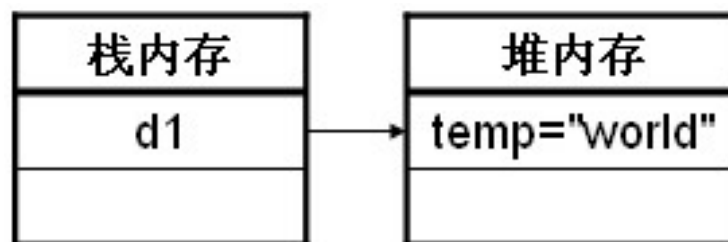
第3种引用传递

```
class Demo {  
    String temp = "hello";    // 此处为了访问方便，属性暂不封装  
}  
  
public class RefDemo03 {  
    public static void main(String[] args) {  
        Demo d1 = new Demo();    // 实例化对象  
        d1.temp = "world";    // 修改对象中的temp属性  
        System.out.println("fun()方法调用之前: " + d1.temp);  
        fun(d1);    // 调用fun()方法  
        System.out.println("fun()方法调用之后: " + d1.temp);  
    }  
    public static void fun(Demo d2) { // 此处的方法由主方法直接调用  
        d2.temp = "MLDN";    // 修改属性的内容  
    }  
}
```

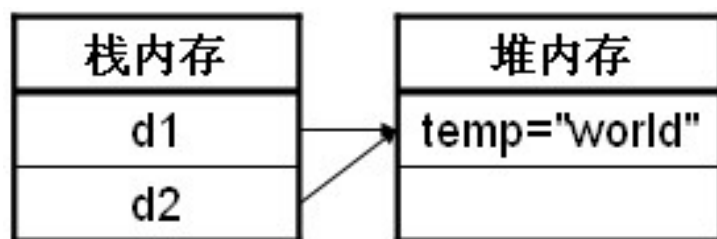
内存分析



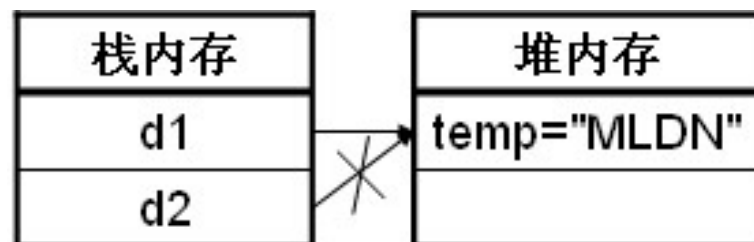
`Demo d1 = new Demo();`



`d1.temp = "world";`



`fun(d1);`



`fun()`方法执行完后断开连接

 **this**



this

this可表示**本类中的方法、属性、构造方法以及当前对象**

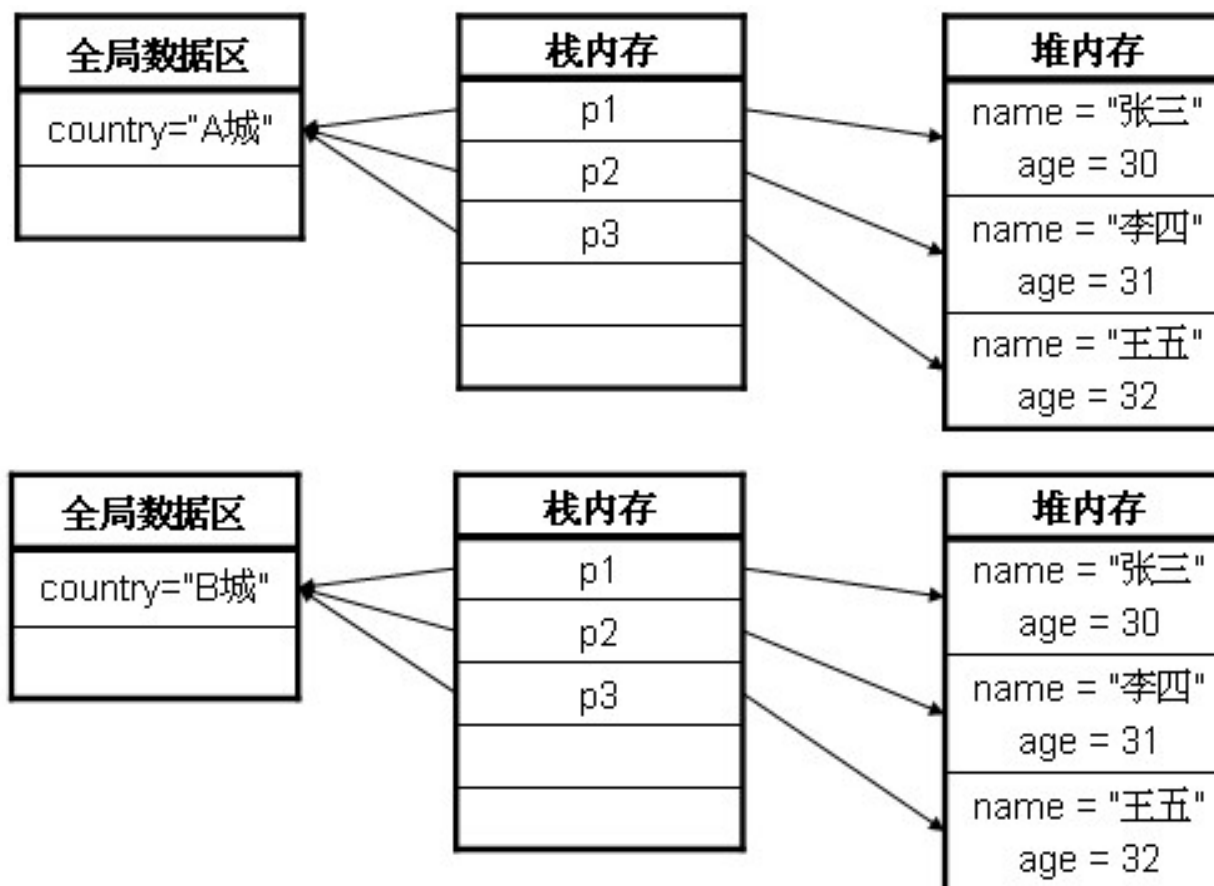
- this.方法() //本类中的成员方法
- this.属性() //本类中的成员变量
- this() //本类中的构造方法，注意此代码应放在第一行
- **表示当前对象，主要用于多个对象进行比较。**

static

- 在程序中使用static声明的**属性**被称为**全局/静态属性**

```
class Person {  
    String name;           // 定义name属性, 此处暂不封装  
    int age;               // 定义age属性, 此处暂不封装  
    String country = "A城"; // 定义城市属性, 有默认值  
    public Person(String name, int age) { // 通过构造方法设置name和age  
        this.name = name;           // 为name赋值  
        this.age = age;             // 为age赋值  
    }  
    public void info() {           // 直接打印信息  
        System.out.println("姓名: " + this.name + ", 年龄: " + this.age + ", 城市  
: "+ country);  
    }  
}
```

Static-内存分析



- Static修饰的方法，被称为“**类方法**”。

```
class Person {  
    private String name;           // 定义name属性  
    private int age;              // 定义age属性  
    private static String country = "A城"; // 定义static属性  
    public static void setCountry(String c) { // 定义static方法  
        country = c;              // 修改static属性  
    }  
    ...  
}
```



static

- 非static声明的方法可以调用**static**和**非static**声明的属性或方法。
- static声明的方法只能调用**static**声明的属性或方法。

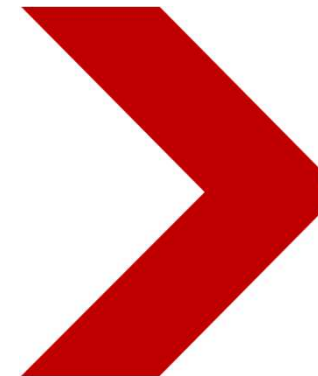
理解main方法

- `public static void main(String args[])`
 - **public**: 表示此方法可以被外部所调用。
 - **static**: 表示此方法可以由类名称直接调用。
 - **void**: 主方法是程序的起点，所以不需要任何的返回值。
 - **main**: 系统规定好默认调用的方法名称。
 - **String args[]**: 表示的是运行时的参数。
- 参数传递的形式: `java 类名称 参数1 参数2 参数3 ....`

Java程序设计基础

Java Programming Fundamentals

Classes and Objects(3)





- 代码块
- 构造方法私有化
- 对象数组
- 内部类

代码块

- 代码块是指使用 “{}” 括起来的一段代码
- 根据位置不同，可分为四种
 - 普通代码块
 - 构造块
 - 静态代码块
 - 同步代码块

普通代码块

- 直接在方法或是语句中定义的代码块

```
public class CodeDemo01 {  
    public static void main(String args[]) {  
        {                                // 定义一个普通代码块  
            int x = 30 ;                // 定义局部变量  
            System.out.println("普通代码块 --> x = " + x);  
        }  
        int x = 100 ;                  // 与局部变量名称相同  
        System.out.println("代码块之外 --> x = " + x);  
    }  
};
```

构造块

- 直接写在类中的代码块。

```
class Demo {  
    {                // 定义构造块  
        System.out.println("1、构造块。") ;  
    }  
    public Demo () {    // 定义构造方法  
        System.out.println("2、构造方法。") ;  
    }  
}  
  
public class CodeDemo02 {  
    public static void main(String args[]) {  
        new Demo () ; // 实例化对象  
        new Demo () ; // 实例化对象  
        new Demo () ; // 实例化对象  
    }  
}
```

//构造块优先于构造方法执行，且每次实例化对象时都会执行。

静态代码块

• 使用static关键字声明的代码块。

```
class Demo {  
    {                                     // 定义构造块  
        System.out.println("1、构造块。") ;  
    }  
    static{                             // 定义静态代码块  
        System.out.println("0、静态代码块") ;  
    }  
    public Demo () {                    // 定义构造方法  
        System.out.println("2、构造方法。") ;  
    }  
} //静态代码块优先于主方法执行，而在类中定义的静态代码块会优先于构造块执行，且只执行一次  
public class CodeDemo03 {  
    static{                             // 在主方法所在类中定义静态块  
        System.out.println("在主方法所在类中定义的代码块。") ;  
    }  
    public static void main(String args[]) {  
        new Demo () ;                 // 实例化对象  
        new Demo () ;                 }  
};
```

同步代码块

- 同步代码块 (synchronized block) 是一种用于实现多线程同步的机制

synchronized(对象锁) {

 // 需要同步的代码

}

increment() 方法使用了同步方法,
而 incrementWithBlock() 方法则使用了同步代码块。两者都能保证 count++ 操作的线程安全性, 但同步代码块提供了更大的灵活性。

```
public class Counter {  
    private int count = 0;  
  
    public synchronized void increment() {  
        count++;  
    }  
  
    public void incrementWithBlock() {  
        synchronized (this) {  
            count++;  
        }  
    }  
}
```



单例模式

- 通过将构造函数私有化的方式实现

构造方法私有化

- 类的封装性不光体现在对属性的封装上，方法也可以被封装，也包括对构造方法的封装。
- 例如：以下代码，封装了构造方法。

```
class Singleton {  
    private Singleton() {                // 此处将构造方法进行封装  
    }  
    public void print() {                // 打印信息  
        System.out.println("Hello World!!!");  
    }  
}
```

单例模式

- 单例模式是一种对象创建模式，用于生产一个对象的实例，它可以确保**系统中一个类只产生一个实例**
 - 对于频繁使用的对象，可以省略创建对象所花费的时间，这对于那些重量级对象而言，是非常可观的一笔系统开销。
 - 由于new操作的次数减少，所以系统内存的使用率也会降低，这将减少GC压力，缩短GC停顿时间。
 - 单例模式的使用对于系统的关键组件和频繁使用的对象来说是可以有效的改善系统的性能的。
- 单例的实现是**通过一个方法返回唯一的一个对象实例**。
 - 单例类必须有一个private访问级别的构造函数，因为，只有这样，才能保证单例不会再系统中的其他代码内被实例化，
 - instance成员变量和getInstance方法必须是static的。

示例

```
public class Test {  
    public static void main(String[] args) {  
        Singleton s1=Singleton.getInstance();  
        Singleton s2=Singleton.getInstance();  
        System.out.println(s1);  
        System.out.println(s2);  
    }  
}  
class Singleton {  
    private static Singleton instance=new Singleton();//设置属性  
    public static Singleton getInstance() { //获取属性，得到单例对象  
        return instance;  
    }  
    private Singleton() { // 此处将构造方法进行封装  
    }  
    public void print(){ // 打印信息  
        System.out.println("Hello World!!!");  
    }  
}
```